Project: Trainen van een NN on MNIST
Geavanceerde computerarchitecturen

Jan Martens & Leon Cherchye

December 15, 2024



Session Date: 29 november - 15 december, 2024

Class: Ma ELICT optie software-engineering

1 Introductie

In dit project onderzochten we hoe we de GPU efficiënt konden gebruiken voor het trainen van een neuraal netwerk (NN). We implementeerden zowel de forward-pass als de backward-pass parallel op de GPU, waarbij de gewichten en biases van de neuronen werden aangepast.
Daarnaast optimaliseerden we het trainingsproces van het netwerk om de trainingsduur te minimaliseren en vergeleken we de prestaties van de GPU met die van de CPU op basis van snelheid.

2 Ons neuraal netwerk

Ons neuraal netwerk bestaat uit een inputlaag van 28x28 neuronen, twee verborgen lagen van respectievelijk 256 en 128 neuronen, en een outputlaag met 10 neuronen. Alle lagen zijn volledig verbonden, wat betekent dat elk neuron in een laag verbinding heeft met alle neuronen in de aangrenzende lagen. Een grafische voorstelling van het NN kan je zien in Figure 2.

3 De dataset

We gebruikten de MNIST dataset, die bestaat uit grijswaardenafbeeldingen van handgeschreven cijfers van 28x28 pixels. Voor elke afbeelding is er een bijbehorend target, een binaire vector van lengte 10 die aangeeft welk cijfer de afbeelding vertegenwoordigt.

4 Trainen van het netwerk

Een neuraal netwerk wordt getraind door de output te vergelijken met de targetwaarden en de fouten (gradiënten) via de backward-pass terug te sturen om de gewichten en biases aan te passen. In dit project gebruikten we de GPU om zowel de forward-pass als de backward-pass parallel uit te voeren, wat de trainingssnelheid aanzienlijk verhoogt.

4.1 Forward pass

De forward-pass berekent de output van het netwerk door de inputs door de verschillende lagen van het netwerk te propageren. Elke node zijn waarde wordt berekent door de gewichten van alle inkomende connecties te vermenigvuldigen met de waardes van de nodes waarmee de connecties verbonden zijn en deze dan op te tellen. Daarbij wordt dan nog een bias opgeteld en deze waarde wordt als input gebruikt voor een activatiefunctie (in dit geval de sigmoidfunctie) om de output van de neuronen te berekenen. Wiskundig ziet dit eruit zoals voorgesteld in Figure 1.

4.2 Backward pass

De backward-pass berekent de gradiënten van de fouten met respect tot de gewichten en biases. Deze gradiënten worden vervolgens gebruikt om de gewichten en biases bij te werken, waarbij de gradiente-daling wordt toegepast met een specifieke leersnelheid.

5 Processing the inputdata

De inputgegevens zijn afbeeldingen van 28x28 pixels, die worden omgezet in een platte array van 784 elementen met waarden tussen 0 en 255. Deze arrays worden in één keer geladen in het

GPU-geheugen per epoch. Eveneens worden de target-gegevens als binaire arrays opgeslagen en geladen in het GPU-geheugen voor elke inputafbeelding.

6 Optimalisaties

6.1 Streaming en coalesced memory-access

Vanwege de aard van de gegevens kunnen we de geoptimaliseerde geheugentoegang via streaming niet volledig benutten. De berekeningen die gebeuren hebben namelijk invloed op gedeelde gegevens die in elke thread worden aangepast. Alle backward-passes moeten dus sequentieel uitgevoerd worden, wat ervoor zorgt dat alle threads sowieso op elkaar moeten wachten. Dit hebben we wel kunnen optimaliseren door de backward-pass en het updaten van de gewichten en biases op te splitsen waardoor de kernel die gedeelde gegevens aanpast zo klein mogelijk is. Coalesced memory accesses zorgen er wel voor dat geheugentoegang efficiënt gebeurd. Alle voor éénzelfde afbeelding zit namelijk sequentieel in het geheugen, en de verschillende afbeeldingen worden in de volgord waarin ze zijn opgeslagen verwerkt.

6.2 Thread divergence

In dit geval is thread divergence niet van toepassing, omdat de bewerkingen op elke thread onafhankelijk zijn en er geen takken zijn die verschillende paden volgen, wat typisch is voor thread divergence. Elke thread zal dus ongeveer even lang doen over zijn berekeningen. Echter moeten alle threads op het einde sequentieel behandeld worden, dus zou thread-divergence niet zoveel problemen geven.

6.3 Inladen data naar GPU

Om de verwerking van de input-data efficiënter te laten verlopen, probeerden we te werken met een soort van 'batches' waarin we de input data van een aantal afbeeldingen in één keer kopieerden naar het GPU-geheugen. Dit had echter geen invloed op de uitvoeringstijd van het programma zoals te zien in Figure 3. De verschillen in de uitvoeringstijd zijn volgens ons te verklaren door de kleine sample-grootte en het feit dat de GPU op het moment van testen soms ook door andere processen werd gebruikt.

6.4 Aantal threads/block

Het aantal threads per block is optimaal ingesteld op 256, wat overeenkomt met de GPU-architectuur die 896 cores en 14 streaming multiprocessors heeft. Dit resulteert in een warp-grootte van 64 cores, wat de parallelisatie efficiënt maakt.

7 Vergelijking met de CPU

De GPU is trager dan de CPU in het geval van een klein netwerk. Dit is te verklaren door de overhead die optreed bij het kopiëren van de data naar de GPU die niet opweegt tegen het voordeel dat gehaald wordt uit de parallellisatie door de GPU. Hoe breder de hidden layers zijn, hoe meer threads er parallel het NN kunnen doorlopen, wat resulteert in een grotere tijdswinst. De CPU moet voor elke node namelijk ook parallel de berekeningen doen wat veel meer berekeningen zijn bij brede lagen.

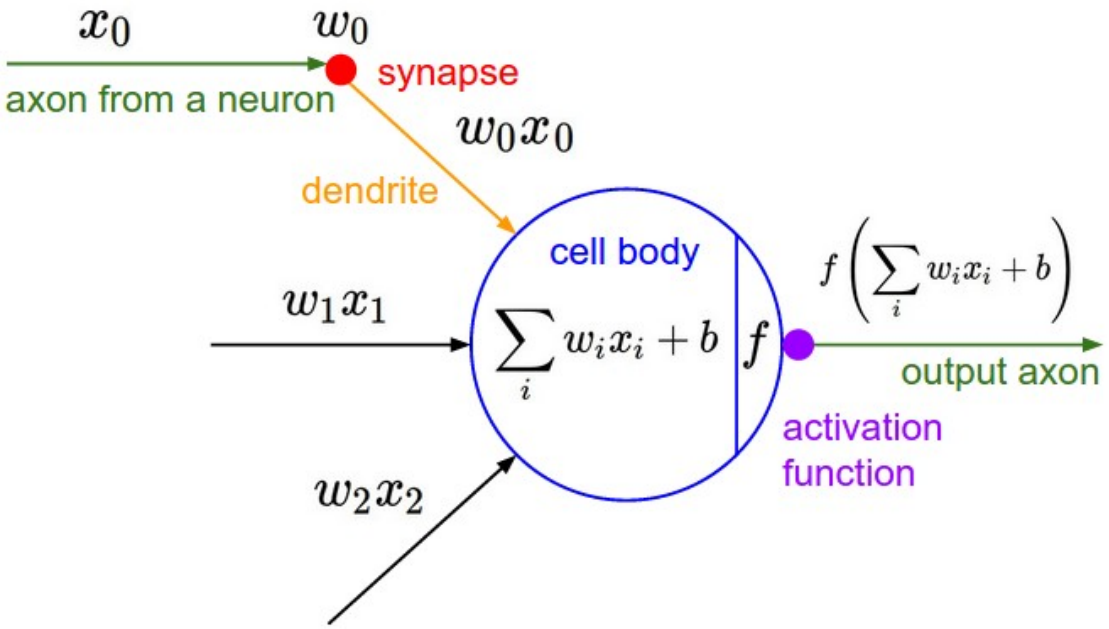


Figure 1: Structuur van één neuron

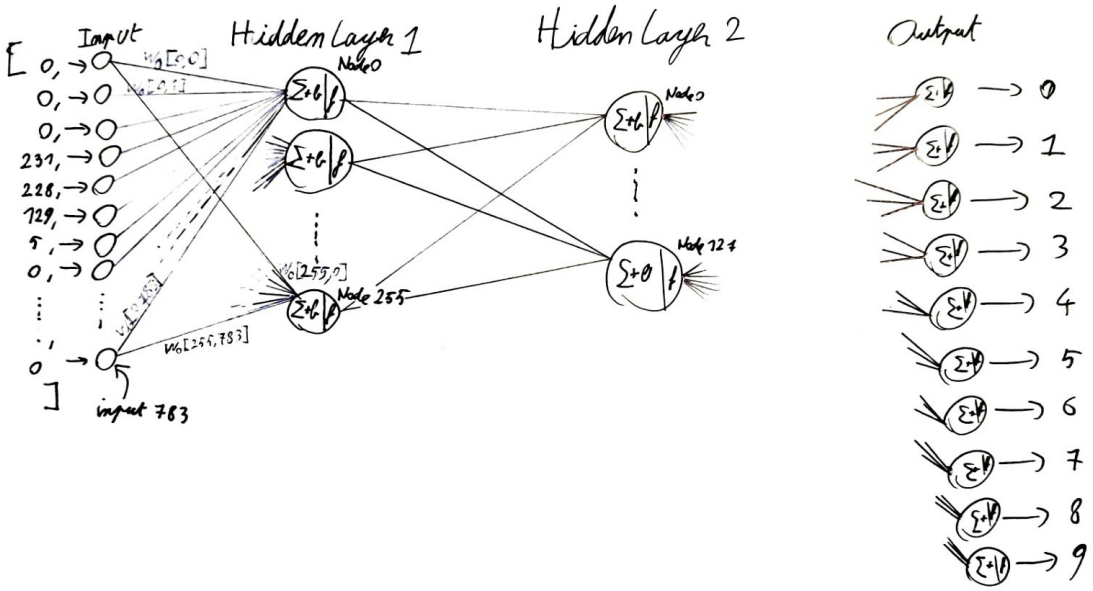


Figure 2: Structuur van het NN

Breedte Hidden Layers	Tijd CPU, 1 core (seconden)	Tijd GPU (seconden)	Aantal keer GPU sneller
8	2.066	7.435	0.28
16	4.090	7.608	0.54
32	8.265	8.122	1.02
64	17.315	8.464	2.05
128	36.901	12.432	2.97
256	83.984	21.469	3.91
512	207.096	27.518	7.53
1024	577.579	39.597	14.59
2048	1760.147	63.303	27.81
4096	6480.660	166.535	38.96

Table 1: Vergelijking van tijd op CPU en GPU voor verschillende breedtes van hidden layers: 3 epochs

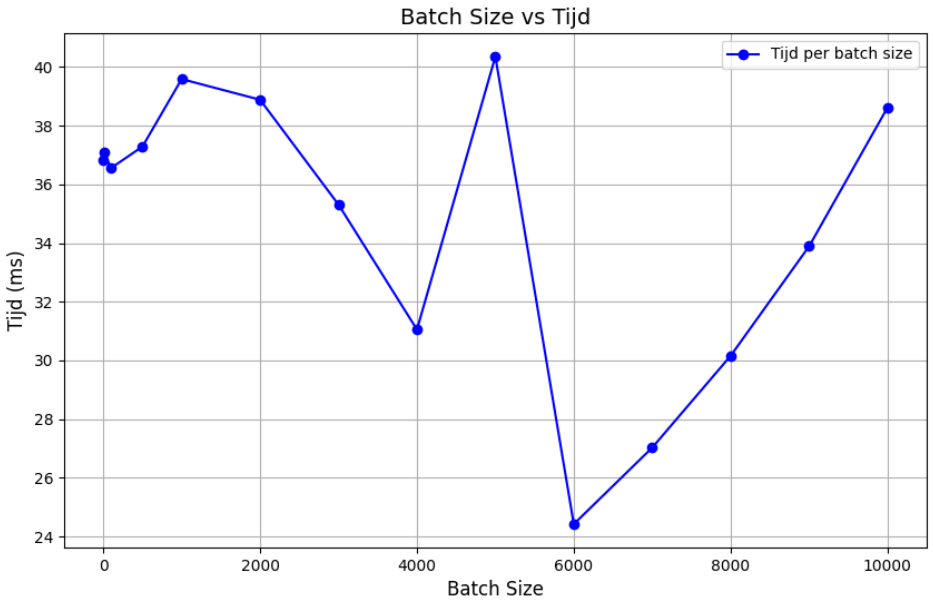


Figure 3: batch-grootte vs uitvoeringstijd 1 epoch